

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
GRADUAÇÃO EM CLOUD COMPUTING
CLOUD SECURITY: AUTOMATION E DEVSECOPS

FERRAMENTAS OPEN SOURCE DE SAST, SCA, IaC SECURITY E DAST NO PIPELINE

*Check Point 01: análise comparativa e aplicação prática de Semgrep, OWASP
Dependency-Check, Checkov e OWASP ZAP*

Anderson Huang, RM 565920

Bruno Henrique, RM 566277

Guyllherme Miguel, RM 562374

Luiz Brito, RM 562192

São Paulo
2026

Sumário

1. Introdução	4
2. As quatro frentes de teste automatizado	4
3. Semgrep (SAST)	5
a. Identificação	5
b. Fundamento técnico	5
c. Instalação e uso	6
d. Integração	7
e. Avaliação crítica	7
4. OWASP Dependency-Check (SCA)	8
a. Identificação	8
b. Fundamento técnico	9
c. Instalação e uso	9
d. Integração	10
e. Avaliação crítica	10
5. Checkov (IaC Security)	12
a. Identificação	12
b. Fundamento técnico	13
c. Instalação e uso	13
d. Integração	15
e. Avaliação crítica	15
6. OWASP ZAP (DAST)	16
a. Identificação	16
b. Fundamento técnico	17
c. Instalação e uso	17
d. Integração	18
e. Avaliação crítica	19
7. Seções transversais	20
7.1 SAST não é SCA	20
7.2 Posicionamento no pipeline	20
7.3 Shift-left não elimina o DAST	21

7.4 SBOM	21
7.5 Quadro comparativo	22
8. Conclusão	22
9. Referências	23
Fontes primárias	23
Fontes secundárias	24

1. Introdução

Segurança de aplicações deixou de ser uma etapa posterior ao desenvolvimento. O modelo DevSecOps distribui a verificação ao longo de todo o ciclo, de forma que cada estágio do pipeline tenha um teste automatizado correspondente. Este trabalho investiga quatro categorias dessas verificações e aplica duas delas em um laboratório executável.

A pergunta que orienta o documento é prática: o que cada ferramenta consegue ver, o que ela não consegue, e em que ponto do pipeline ela precisa estar para que o defeito seja encontrado antes de chegar à produção.

O recorte do grupo é a toolchain OWASP em um pipeline tradicional: Semgrep para SAST, OWASP Dependency-Check para SCA, Checkov para IaC Security e OWASP ZAP para DAST. As quatro são open source, com atividade recente de desenvolvimento e integração pronta para GitHub Actions.

O laboratório conduzido para a turma aplica Checkov e OWASP ZAP, duas categorias diferentes, cada uma com uma vulnerabilidade real e uma correção de uma linha.

2. As quatro frentes de teste automatizado

As quatro categorias se distinguem por três critérios: o artefato analisado, o estágio do pipeline e a necessidade de a aplicação estar em execução.

Categoria	Analisa	Estágio	Aplicação no ar
SAST	Código-fonte próprio (white-box)	Code, Build	Não
SCA	Dependências de terceiros e licenças	Build	Não
IaC Security	Terraform, CloudFormation, Kubernetes, Docker, Helm	Code, Build, Deploy	Não
DAST	Aplicação em execução (black-box)	Test, Release	Sim

A confusão mais comum é entre SAST e SCA, e a distinção é objetiva: SAST analisa o código que a equipe escreveu, SCA analisa o código que a equipe importou. São defeitos de naturezas diferentes, com formas de correção diferentes. Uma falha de SAST se corrige reescrevendo uma linha; uma falha de

SCA se corrige atualizando uma versão, ou substituindo a biblioteca quando não há correção disponível.

O IaC Security aparece duas vezes no pipeline de propósito. O mesmo manifesto é analisado quando é escrito e verificado de novo antes de provisionar, porque um Terraform aprovado no commit pode gerar um recurso inseguro no deploy se uma variável de ambiente mudar.

O DAST é a única das quatro que exige a aplicação em execução, e é por isso que ele não desaparece com a adoção de shift-left. SAST, SCA e IaC analisam artefatos parados: não veem erro de configuração de runtime, falha de sessão autenticada nem comportamento da aplicação integrada.

3. Semgrep (SAST)

a. Identificação

Item	Valor
Ano de origem	2020, derivado do <code>sgrep</code> , ferramenta interna criada no Facebook
Mantenedor	Semgrep, Inc. (antiga r2c)
Licença	LGPL-2.1
Linguagem	OCaml, com interface em Python
Repositório	<code>semgrep/semgrep</code> , criado em 13/12/2019

Atividade do projeto, consultada pela API do GitHub em 28/09/2026: 16.786 estrelas, 1.073 forks, 205 contribuidores e 1.027 commits nos últimos doze meses. A versão estável mais recente é a v1.178.0, publicada em 23/09/2026. A versão usada nas medições foi a 1.177.0.

O projeto tem o ritmo de commits mais alto das quatro ferramentas analisadas, com mais de mil commits em doze meses.

b. Fundamento técnico

O Semgrep faz análise estática por **correspondência de padrões sobre a árvore sintática abstrata** (AST). Em vez de comparar texto, ele converte o código em uma árvore e busca padrões escritos na própria sintaxe da linguagem alvo.

Isso significa que um padrão como `res.send(...)` casa com todas as formas equivalentes de escrever aquela chamada, independentemente de espaçamento,

quebras de linha ou nomes de variáveis intermediárias. A regra é escrita parecida com o código que ela procura, o que diminui a barreira para criar regras próprias.

A versão de código aberto também realiza análise de fluxo de dados dentro de uma mesma função, o que permite rastrear um valor da entrada até o ponto de uso.

O que detecta: padrões inseguros no código-fonte próprio, como concatenação de entrada do usuário em resposta HTTP, uso de funções criptográficas fracas, credenciais embutidas no código e ausência de middlewares de proteção.

O que não detecta: vulnerabilidades em dependências de terceiros, que são domínio do SCA; falhas que só existem em runtime, como erro de configuração de servidor; e falhas de lógica de negócio. A análise de fluxo entre arquivos diferentes não está disponível na versão open source.

c. Instalação e uso

A execução do grupo usou a imagem Docker oficial, sem instalação local:

```
docker run --rm -v "$PWD/lab":/src -w /src semgrep/semgrep:latest \
  semgrep scan --config auto --json-output=/src/semgrep.json app
```

Flag	Função
<code>--config auto</code>	Baixa o conjunto de regras adequado às linguagens detectadas
<code>--config p/ <pacote></code>	Usa um pacote específico do registro público de regras
<code>--json-output</code>	Grava o resultado em JSON
<code>--sarif-output</code>	Grava em SARIF, formato aceito pelo GitHub Code Scanning
<code>--error</code>	Encerra com código diferente de zero quando há achado, usado como gate

Supressão de falso positivo. O Semgrep aceita o comentário `nosemgrep` na linha anterior ao achado, opcionalmente com o identificador da regra, o que limita a supressão àquela regra específica.

Regras próprias. As regras são escritas em YAML e o padrão usa a sintaxe da linguagem alvo, com metavariáveis em maiúsculas para representar trechos variáveis. Uma regra que detecte o defeito da aplicação do laboratório teria esta forma:

```
rules:
  - id: express-send-user-input
    patterns:
      - pattern: $RES.send(... $REQ.query.$P ...)
    message: Entrada do usuario enviada na resposta sem escape
    languages: [javascript]
    severity: WARNING
```

O padrão descreve o código procurado quase como ele é escrito, o que distingue o Semgrep de ferramentas cujas regras exigem conhecer a representação interna da AST.

d. Integração

O Semgrep gera SARIF nativamente, o que permite enviar os achados ao GitHub Code Scanning e vê-los na aba Security do repositório, anotados na linha exata do código. O pipeline anterior do grupo usava exatamente essa integração:

```
- name: semgrep-scan
  run: semgrep scan --config auto --error --sarif-output=semgrep.sarif app
- name: upload-sarif
  uses: github/codeql-action/upload-sarif@v4
```

DefectDojo. O parser "Semgrep JSON Report" consome o relatório em JSON, formato produzido pela flag `--json-output`. A verificação foi feita lendo o código do parser no repositório do DefectDojo.

IDE e pre-commit. A ferramenta roda como hook de pre-commit e tem extensões de IDE, o que a coloca no estágio mais à esquerda possível do pipeline: o defeito pode ser apontado antes mesmo do commit.

e. Avaliação crítica

Medição do grupo. A execução contra `lab/app` levou **84 segundos**, rodou **223 regras** sobre **6 arquivos** e produziu **2 achados**.

Regra	Linha	Severidade	CWE
<code>express.security.audit.xss.direct-response-write</code>	<code>index.js:9</code>	WARNING	CWE-79
<code>express.security.audit.express-check-csrf-middleware-usage</code>	<code>index.js:3</code>	INFO	CWE-352

O primeiro achado é o resultado mais significativo do trabalho: **o Semgrep encontrou, por análise estática, exatamente a mesma vulnerabilidade que o**

OWASP ZAP encontrou atacando a aplicação em execução. São caminhos independentes chegando ao mesmo CWE-79, na mesma linha.

A diferença está no custo e no momento. O Semgrep apontou a linha 9 do arquivo em 84 segundos, sem subir a aplicação. O ZAP precisou da aplicação no ar, de um spider para descobrir a rota e de um scan ativo com injeção de payloads, e ainda assim não informa em que linha está o defeito.

Taxa de falso positivo observada. Nenhum dos dois achados é falso positivo. O de CWE-79 foi confirmado por execução real. O de CWE-352 aponta a ausência de proteção contra CSRF, correta para uma aplicação com rotas que alteram estado, ainda que a aplicação do laboratório só tenha rotas GET. A severidade INFO atribuída pela ferramenta é adequada a essa ressalva.

Limitações encontradas. O `--config auto` depende de conexão com o registro de regras da Semgrep, o que torna a execução dependente de rede e do serviço estar disponível. Para um pipeline determinístico, convém fixar um pacote de regras específico.

Cenário ideal. Verificação em todo pull request e como hook de pre-commit, no estágio Code do pipeline, com envio dos achados ao Code Scanning.

Por que escolher esta. Entre as opções de SAST open source, o Semgrep foi escolhido por três razões medidas no trabalho. A primeira é o custo de execução: 84 segundos contra 223 regras, tempo compatível com a verificação em todo pull request. A segunda é a sintaxe de regras, que permite ao grupo escrever verificações próprias sem estudar a representação interna da árvore sintática. A terceira é a saída em SARIF nativa, que elimina qualquer conversão intermediária para chegar ao Code Scanning.

4. OWASP Dependency-Check (SCA)

a. Identificação

Item	Valor
Ano de origem	2012
Mantenedor	Jeremy Long e comunidade, sob a OWASP
Licença	Apache-2.0
Linguagem	Java
Repositório	dependency-check/DependencyCheck , criado em 03/09/2012

Atividade do projeto, consultada pela API do GitHub em 28/09/2026: 7.709 estrelas, 323 contribuidores e 520 commits nos últimos doze meses. A versão estável mais recente é a v13.0.0, publicada em 03/08/2026, que foi a usada nas medições.

É o projeto mais antigo da toolchain, com catorze anos de existência.

Cabe um registro de verificação: existe também o repositório `jeremylong/DependencyCheck`, que aparece com frequência em resultados de busca. Trata-se de um fork criado em 2025, com 55 estrelas e nenhum commit nos últimos doze meses. O repositório oficial é o indicado na tabela acima.

b. Fundamento técnico

O Dependency-Check identifica as dependências declaradas pelo projeto e verifica se existem vulnerabilidades conhecidas associadas a elas. O processo tem duas etapas.

Primeiro, a ferramenta extrai evidências de cada arquivo analisado e tenta associá-las a um identificador padronizado de produto, o CPE (Common Platform Enumeration). Depois, consulta a base da NVD por vulnerabilidades registradas para aquele CPE.

Essa dependência do CPE é a origem da principal fragilidade da ferramenta: a associação é heurística, o que produz tanto falsos positivos, quando o nome de uma biblioteca coincide com o de outro produto, quanto falsos negativos, quando a associação não é feita.

O que detecta: vulnerabilidades publicamente conhecidas em bibliotecas de terceiros, com identificador CVE e pontuação CVSS.

O que não detecta: vulnerabilidades no código escrito pela equipe, que são domínio do SAST; falhas ainda não publicadas em base pública; e dependências que não estejam declaradas nos formatos que a ferramenta reconhece.

c. Instalação e uso

A execução do grupo usou a imagem Docker oficial:

```
docker run --rm \
-v "$PWD/app":/src -v "$PWD/dc-out":/report \
-v "$PWD/dc-data":/usr/share/dependency-check/data \
owasp/dependency-check:latest \
--scan /src --format JSON --format HTML --project "lab-app" --out /report \
--nvdApiKey "<chave>"
```

Flag	Função
<code>--scan</code>	Caminho a ser analisado
<code>--format</code>	Formato do relatório: HTML, JSON, XML, CSV, SARIF, JUNIT ou ALL
<code>--out</code>	Diretório de saída
<code>--nvdApiKey</code>	Chave de acesso à API da NVD
<code>--failOnCVSS</code>	Encerra com código diferente de zero acima da pontuação CVSS indicada, usado como gate
<code>--suppression</code>	Arquivo XML de supressão de falso positivo
<code>--noupdate</code>	Não atualiza a base, usando apenas o cache local

O volume montado em `/usr/share/dependency-check/data` é o que preserva a base entre execuções. Sem ele, a base é baixada novamente a cada scan.

Supressão de falso positivo. É feita por arquivo XML, no qual cada supressão declara a dependência afetada e o CVE a ignorar, com justificativa.

d. Integração

CI/CD. Há uma GitHub Action oficial e plugins para Maven, Gradle, Ant e Jenkins.

SARIF. A ferramenta gera SARIF desde a versão 6, o que permite envio ao GitHub Code Scanning.

DefectDojo. O parser "Dependency Check Scan" consome o relatório em **XML**, verificado por leitura do código do parser, que usa `ElementTree`. Como o laboratório gera JSON e HTML, a importação exigiria adicionar `--format XML`.

IDE e pre-commit. Não há extensão de IDE mantida pelo projeto. O uso como hook de pre-commit é inviável pelo tempo de execução e pelo tamanho da base local.

e. Avaliação crítica

Esta foi a ferramenta que exigiu mais tentativas até produzir um resultado válido, e as falhas do caminho são o material mais útil desta seção.

Primeira execução: sem chave da NVD. A ferramenta encerrou com erro fatal, sem gerar relatório:

```
NvdApiException: Invalid API Key, length of 0 too short to provided a masked
partial key
[ERROR] Error updating the NVD Data
```

A chave é gratuita, mas exige cadastro prévio. Esta é uma diferença relevante em relação às outras três ferramentas da toolchain, que executam sem qualquer credencial.

Segunda execução: com chave, sem lock file. A ferramenta rodou, mas analisou apenas 2 arquivos e relatou 0 vulnerabilidades. O resultado parece limpo e não é: nenhuma dependência havia sido resolvida.

Terceira execução: com lock file, sem node_modules. Ainda insuficiente. O log foi explícito:

```
[WARN] Analyzing `src/package-lock.json` - however, the node_modules
directory
does not exist. Please run `npm install` prior to running dependency-check
```

Quarta execução: com npm install executado. Resultado válido: **265 dependências analisadas, 0 vulnerabilidades encontradas.**

A observação crítica é que as execuções 2 e 3 produziram relatórios de aparência normal, com zero vulnerabilidades, sem qualquer erro visível no resultado final. Um pipeline configurado dessa forma passaria no gate indefinidamente sem nunca ter analisado uma única biblioteca. **Zero achados não significa ausência de vulnerabilidade; pode significar ausência de análise.**

Tempos medidos.

Etapa	Tempo
Download inicial da base da NVD, 398.697 registros	34 min 34 s
Execução com a base em cache	9 s

A diferença entre a primeira execução e as seguintes é de mais de duzentas vezes. Em pipeline, isso torna o cache da base um requisito, não uma otimização.

Falha de rede durante o download. Uma das tentativas foi interrompida em 40% com `java.net.UnknownHostException: www.cisa.gov: Name does not resolve`, pois a ferramenta também consulta a lista de vulnerabilidades exploradas conhecidas da CISA e o repositório do RetireJS. O download precisou recomeçar do zero.

Analizador desabilitado por falta de credencial. O log registrou que o Sonatype OSS Index foi desativado por ausência de token, o que reduz o conjunto de fontes consultadas sem que isso apareça no relatório final.

Taxa de falso positivo observada. Não foi possível medir, pois o scan válido não produziu nenhum achado. As dependências do laboratório são Express 5.2.1 e Helmet 8.3.0, ambas em versões recentes e sem CVE registrada. Registrar isso é mais honesto do que apresentar uma taxa que não foi observada.

Esse resultado ilustra uma característica do SCA: seu valor é proporcional ao tamanho e à idade da árvore de dependências. Em uma aplicação com duas dependências diretas e recentes, a ferramenta corretamente não encontra nada.

Limitações encontradas. A necessidade de chave da NVD, o tempo de download inicial, a exigência de dependências efetivamente instaladas e a dependência de heurística de CPE fazem desta a ferramenta de operação mais custosa entre as quatro analisadas.

Cenário ideal. Estágio Build do pipeline, após a instalação de dependências, com a base da NVD em cache persistente e gate configurado por `--failOnCVSS`.

Por que escolher esta. É a ferramenta de SCA da própria OWASP, com catorze anos de projeto e 520 commits nos últimos doze meses. Gera SBOM em CycloneDX e relatório HTML legível sem ferramenta adicional. A ressalva é operacional: exige credencial, cache e dependências instaladas, três condições que precisam estar explícitas no pipeline para que o resultado signifique alguma coisa.

5. Checkov (IaC Security)

a. Identificação

Item	Valor
Ano de origem	2019
Mantenedor	Bridgecrew, adquirida pela Palo Alto Networks em 2021
Licença	Apache-2.0
Linguagem	Python
Repositório	<code>bridgecrewio/checkov</code> , criado em 27/11/2019

Atividade do projeto, consultada pela API do GitHub em 28/09/2026: 9.035 estrelas, 1.422 forks, 381 contribuidores e 239 commits nos últimos doze meses. A versão estável mais recente é a 3.3.20, publicada em 27/09/2026, que foi a usada nas medições.

b. Fundamento técnico

O Checkov faz análise estática de infraestrutura como código. Ele converte o manifesto em um grafo de recursos e avalia políticas sobre esse grafo, o que permite escrever regras que dependem da relação entre recursos, e não apenas dos atributos de um recurso isolado.

Suporta Terraform, CloudFormation, Kubernetes, Helm, Dockerfile, Serverless Framework e ARM, entre outros formatos.

O que detecta: configurações inseguras declaradas no manifesto, como portas administrativas abertas para a internet, armazenamento sem criptografia, ausência de logs e permissões excessivamente amplas.

O que não detecta: o estado real da infraestrutura provisionada. O Checkov lê o arquivo, não a nuvem. Um recurso criado manualmente, ou alterado fora do Terraform, é invisível para ele. Também não avalia valores que só são resolvidos em tempo de execução, vindos de variáveis de ambiente ou de fontes de dados remotas.

c. Instalação e uso

O laboratório usa a imagem oficial com a tag fixada:

```
docker compose run --rm checkov
```

Por trás, o comando executado é:

```
checkov -d /iac --compact -o cli -o github_failed_only --output-file-path  
console,/reports/checkov.md
```

Flag	Função
<code>-d, --directory</code>	Diretório a ser analisado, de forma recursiva
<code>--compact</code>	Omite o bloco de código na saída, mantendo apenas o veredito
<code>-o, --output</code>	Formato de saída: cli, csv, json, junitxml, sarif, github_failed_only, entre outros
<code>--output-file-path</code>	Destino dos relatórios, mapeado posicionalmente aos <code>-o</code>
<code>-s, --soft-fail</code>	Executa as verificações mas sempre retorna código zero, desligando o gate
<code>--hard-fail-on</code>	Retorna código diferente de zero apenas para as verificações indicadas

A combinação `console,/reports/checkov.md` mapeia posicionalmente: a primeira saída (`cli`) vai para o terminal e a segunda (`github_failed_only`) para o arquivo. É assim que o laboratório mantém a saída colorida visível durante a execução e ao mesmo tempo grava o relatório versionado.

Formatos de saída. O Checkov não gera HTML. Dos formatos disponíveis, o `github_failed_only` produz uma tabela Markdown legível, que renderiza diretamente no GitHub. Esta é uma assimetria prática em relação ao ZAP, que entrega HTML pronto com uma única flag.

Supressão de falso positivo. É feita por comentário no próprio manifesto, o que mantém a justificativa versionada junto ao código:

```
resource "azurerm_network_security_group" "lab" {
  # checkov:skip=CKV_AZURE_10:regra de bastion, acesso restrito por Just-in-Time
}
```

Regras próprias. O Checkov aceita políticas customizadas em Python ou em YAML. A versão em YAML descreve a condição diretamente sobre os atributos do recurso, sem exigir código:

```
metadata:
  id: "CKV_CUSTOM_1"
  name: "NSG nao deve liberar SSH em notacao CIDR ampla"
definition:
  cond_type: attribute
  resource_types:
    - azurerm_network_security_group
  attribute: security_rule.source_address_prefix
  operator: not_equals
  value: "0.0.0.0/0"
```

Esta política cobre exatamente o falso negativo descrito na avaliação crítica desta seção, em que a política nativa aprova a notação CIDR.

d. Integração

O Checkov gera SARIF, integrando-se ao GitHub Code Scanning, e roda como hook de pre-commit. Há também extensões de IDE mantidas pelo projeto.

DefectDojo. O parser "Checkov Scan" consome o relatório em JSON, produzido com `-o json`. A verificação foi feita lendo o código do parser no repositório do DefectDojo.

e. Avaliação crítica

Medição do grupo. A execução contra `lab/iac` leva cerca de **20 segundos** e avalia 4 políticas sobre o Network Security Group declarado. No estado vulnerável, o resultado é `Passed checks: 3, Failed checks: 1` com código de saída 1. Após a correção, `Passed checks: 4, Failed checks: 0` com código de saída 0.

Limitação relevante: ausência de severidade. O Checkov open source não classifica os achados por severidade. A documentação oficial é explícita: para filtrar por severidade é necessário executar com a integração da plataforma, via chave de API. Isso significa que o gate do Checkov é binário, ou seja, qualquer política violada quebra o build, sem distinção entre uma configuração crítica e uma recomendação menor.

A consequência prática para este trabalho é direta: o requisito de quebrar o build em severidade alta não pode ser atendido pelo Checkov na versão gratuita. No laboratório, esse papel cabe ao ZAP, que atribui severidade real aos alertas.

Limitação encontrada na prática: a política não reconhece notação CIDR. Durante a construção do laboratório, o grupo escreveu a regra insegura como `source_address_prefix = "0.0.0.0/0"`, que é a forma mais comum de expressar "qualquer origem". O Checkov aprovou o manifesto, com `Passed checks: 4, Failed checks: 0`.

A política `CKV_AZURE_10` só dispara quando o valor é `*` ou `Internet`. Verificamos isso isolando os dois casos:

Valor de <code>source_address_prefix</code>	Resultado	Código de saída
"*"	Failed checks: 1	1
"0.0.0.0/0"	Failed checks: 0	0

As duas configurações expõem a porta 22 à internet inteira. A ferramenta reprova uma e aprova a outra. É um falso negativo real, e o tipo de limitação que só aparece quando se testa a ferramenta em vez de apenas ler sua documentação.

Cenário ideal. Verificação de manifestos de infraestrutura em pull request e novamente antes do apply, no estágio Deploy, dado que valores resolvidos em tempo de execução podem alterar o resultado.

Por que escolher esta. O Checkov cobre a maior variedade de formatos entre as ferramentas de IaC analisadas, o que evita adotar uma ferramenta por tecnologia de infraestrutura. A execução é a mais rápida da toolchain, cerca de 20 segundos, e não depende de rede, ao contrário do Semgrep com `--config auto` e do Dependency-Check com a base da NVD. A ressalva, medida neste trabalho, é que a ausência de severidade na versão gratuita o torna inadequado como gate por criticidade.

6. OWASP ZAP (DAST)

a. Identificação

Item	Valor
Ano de origem	2010, como fork do Paros Proxy
Mantenedor	ZAP Core Team, com apoio da Checkmarx desde setembro de 2024
Licença	Apache-2.0
Linguagem	Java
Repositório	<code>zapproxy/zapproxy</code> , migrado para o GitHub em 03/06/2015

Atividade do projeto, consultada pela API do GitHub em 28/09/2026: 15.839 estrelas, 2.650 forks, 239 contribuidores e 378 commits nos últimos doze meses. A versão estável mais recente é a 2.17.0, publicada em 15/12/2025, que foi a usada nas medições.

O projeto deixou a OWASP em 2023 e integrou o Software Security Project. Desde setembro de 2024 opera como "ZAP by Checkmarx", mantendo a licença Apache-2.0 e o controle técnico com o Core Team.

b. Fundamento técnico

O ZAP é um scanner black-box que funciona como proxy entre o cliente e a aplicação. O fluxo tem três etapas:

1. **Spider:** navega pela aplicação e descobre as URLs a serem testadas. No laboratório, encontrou 5 URLs a partir da raiz.
2. **Regras passivas:** analisam as respostas sem modificar as requisições. Não realizam ataques.
3. **Regras ativas:** reenviam as requisições substituindo os parâmetros por payloads de ataque e avaliam as respostas.

A distinção entre as etapas 2 e 3 determina o que a ferramenta consegue encontrar. **Detectar XSS exige injetar payload**, portanto é atribuição exclusiva do scan ativo. O grupo verificou empiricamente que nenhum alerta de risco alto aparece em uma varredura apenas passiva contra a aplicação do laboratório.

O que detecta: falhas observáveis no comportamento da aplicação em execução, como injeção de SQL, XSS, cabeçalhos de segurança ausentes e vazamento de informação nas respostas.

O que não detecta: a linha de código com o defeito, porque não tem acesso ao código-fonte; rotas que o spider não alcança; páginas atrás de autenticação não configurada; e falhas de lógica de negócio.

c. Instalação e uso

O laboratório usa a imagem `zapproxy/zap-stable:2.17.0` executando um plano do Automation Framework, em vez dos scripts empacotados:

```
docker compose run --rm zap
```

Por trás: `zap.sh -cmd -autorun /zap/zap.yaml`.

Os três modos disponíveis:

Modo	Comando	Comportamento
Baseline	<code>zap-baseline.py -t <url></code>	Spider por 1 minuto e regras passivas, sem ataque
Full scan	<code>zap-full-scan.py -t <url></code>	Spider sem limite e scan ativo completo
Automation Framework	<code>zap.sh -cmd -autorun <plano></code>	Executa os jobs declarados em um plano YAML

Exit codes dos scripts empacotados: 0 para sucesso, 1 para ao menos um FAIL, 2 para ao menos um WARN sem FAIL, 3 para outra falha.

Supressão de falso positivo. O arquivo de configuração passado com `-c` define, por identificador de regra, se o alerta é tratado como WARN, IGNORE ou FAIL. O formato é uma linha por regra:

```
10020 IGNORE (Missing Anti-clickjacking Header)
10096 OUTOFSCOPE http://app:3000/style.css
```

A segunda forma desativa a regra apenas para as URLs que casam com a expressão, mantendo-a ativa no resto da aplicação.

Regras próprias. O ZAP permite escrever regras de varredura passiva e ativa como scripts, em linguagens suportadas pelo motor de scripts, além de aceitar modelos de ataque próprios. Para o escopo deste trabalho, as regras nativas foram suficientes e nenhuma regra própria foi escrita.

d. Integração

O projeto mantém GitHub Actions oficiais para baseline, full scan, API scan e para o Automation Framework. O ZAP gera SARIF por meio do template `sarif-json`, o que permite enviar os achados ao GitHub Code Scanning.

DefectDojo. O parser "ZAP Scan" consome o relatório em **XML**, e não em JSON ou HTML. A verificação foi feita lendo o código do parser, que usa `ElementTree`. Isso tem consequência prática para este trabalho: o plano de automação do laboratório gera HTML e JSON, de modo que a importação para o DefectDojo exigiria adicionar um terceiro job de relatório em XML.

IDE e pre-commit. Não há plugin de IDE, e pre-commit não se aplica: a ferramenta exige a aplicação em execução, condição que não existe nessa etapa.

e. Avaliação crítica

O gate por severidade. Esta foi a decisão técnica mais relevante do laboratório. Os scripts `zap-baseline.py` e `zap-full-scan.py` classificam toda regra como WARN por padrão e encerram com código 2, independentemente de o alerta ser de risco alto ou baixo. O código de saída não carrega a severidade.

O grupo mediu isso: o full scan encontrou três alertas de risco alto e ainda assim encerrou com código 2.

A solução nativa é o job `exitStatus` do Automation Framework, que avalia a severidade real:

```
- type: exitStatus
  parameters:
    errorLevel: High
```

Com esse job, a execução encerra com a mensagem `An alert has been raised with a risk of at least: High` e código de saída 1. O build quebra por severidade, sem reclassificar regras manualmente.

Ganho de tempo medido. A migração dos scripts para o plano de automação também reduziu o tempo de execução, porque o plano declara apenas os jobs necessários:

Execução	Tempo	Código de saída com alerta High
<code>zap-full-scan.py</code>	142 s	2
Plano do Automation Framework	57 s	1

Limitação encontrada na prática: alertas passivos não são determinísticos.

Em execuções consecutivas do mesmo plano, sem qualquer alteração de código, o alerta `10020 Missing Anti-clickjacking Header` apareceu em algumas execuções e não em outras. A causa é o spider, que não alcança necessariamente o mesmo conjunto de URLs a cada execução.

Os dois alertas de XSS, levantados pelo scan ativo, mantiveram-se estáveis em todas as execuções. A conclusão prática é que um gate construído sobre alertas passivos é instável, enquanto um gate sobre achados do scan ativo é confiável.

Taxa de falso positivo observada. Dos alertas levantados contra a aplicação do laboratório, os dois de risco alto são verdadeiros positivos, confirmados por reprodução manual. O alerta de clickjacking, de risco médio, é tecnicamente

correto mas irrelevante no contexto: a aplicação não tem formulário, sessão nem ação com efeito colateral que pudesse ser induzida por sobreposição.

Cenário ideal. Estágio Test do pipeline, em ambiente isolado, com o plano de automação declarando o gate por severidade.

Por que escolher esta. O ZAP é a única ferramenta da toolchain que observa o sistema como ele efetivamente responde, e a única cujo código de saída pode ser condicionado à severidade real do achado, o que o torna o gate de criticidade do pipeline. Some-se a maturidade do projeto, ativo desde 2010, e a existência de imagens Docker e GitHub Actions oficiais, que eliminam trabalho de integração.

7. Seções transversais

7.1 SAST não é SCA

A distinção é o ponto onde a disciplina mais gera confusão, e o laboratório permite demonstrá-la de forma concreta.

	SAST	SCA
Analisa	Código escrito pela equipe	Código importado de terceiros
Pergunta que responde	"Escrevemos algo inseguro?"	"Importamos algo vulnerável?"
Forma de correção	Reescrever o trecho	Atualizar a versão ou substituir a biblioteca
Base de conhecimento	Regras de padrão inseguro	Bases públicas de vulnerabilidades, como a NVD

Na aplicação do laboratório, o Semgrep aponta a linha 9 do `index.js`, escrita pelo grupo. O Dependency-Check examinaria o `package.json`, onde estão Express e Helmet, escritos por terceiros. São dois arquivos, dois tipos de defeito, duas formas de correção.

7.2 Posicionamento no pipeline

O estágio em que cada ferramenta é executada decorre diretamente do artefato que ela analisa.

Estágio	Ferramentas	Artefato disponível
Code	Semgrep, Checkov	Código-fonte e manifestos
Build	Dependency-Check, Checkov	Dependências resolvidas, imagem construída
Test	OWASP ZAP	Aplicação em execução
Deploy	Checkov	Manifesto com valores resolvidos

7.3 Shift-left não elimina o DAST

Shift-left é a prática de antecipar a verificação para os estágios iniciais, onde a correção é mais barata. O argumento é sólido, mas tem limite.

O laboratório demonstra os dois lados. De um lado, o Semgrep encontrou o mesmo XSS que o ZAP, mais rápido, mais cedo e apontando a linha exata. Esse é o argumento a favor do shift-left.

De outro, o Semgrep só encontrou porque o padrão inseguro estava explícito no código. Erro de configuração de servidor, cabeçalho ausente por configuração de proxy, falha de sessão autenticada e comportamento da aplicação integrada não aparecem em nenhum artefato parado. O DAST continua sendo a única categoria que observa o sistema como ele efetivamente responde.

7.4 SBOM

Um SBOM (Software Bill of Materials) é o inventário dos componentes que compõem um software: bibliotecas, versões, licenças e relações de dependência. É a lista de ingredientes do sistema.

A relevância é operacional. Quando uma vulnerabilidade crítica é divulgada em uma biblioteca amplamente usada, a primeira pergunta de qualquer organização é se ela usa aquele componente, e onde. Sem inventário, responder exige varrer todos os repositórios.

O Dependency-Check gera SBOM em CycloneDX, e o Checkov também suporta os formatos CycloneDX e SPDX.

7.5 Quadro comparativo

	Semgrep	Dependency-Check	Checkov	OWASP ZAP
Categoria	SAST	SCA	IaC Security	DAST
Licença	LGPL-2.1	Apache-2.0	Apache-2.0	Apache-2.0
Linguagem	OCaml	Java	Python	Java
Aplicação no ar	Não	Não	Não	Sim
Severidade nativa	Sim	Sim (CVSS)	Não na versão gratuita	Sim
Gera SARIF	Sim	Sim	Sim	Sim (template)
Relatório HTML	Não	Sim	Não	Sim
Pre-commit	Sim	Não	Sim	Não
Tempo medido pelo grupo	84 s	9 s com cache, 34 min sem	20 s	57 s
Depende de rede	Sim (-- config auto)	Sim (base NVD)	Não	Não

8. Conclusão

O resultado mais instrutivo do trabalho não estava previsto no planejamento: Semgrep e OWASP ZAP encontraram a mesma vulnerabilidade, o mesmo CWE-79, por caminhos completamente independentes. Um lendo o código parado, o outro atacando a aplicação em execução. A convergência mostra que as categorias não são alternativas entre si, e sim camadas com custos e momentos diferentes.

As limitações encontradas foram igualmente instrutivas, e nenhuma delas estava na documentação das ferramentas:

- O Checkov aprova 0.0.0.0/0 e reprova * para a mesma exposição de SSH, um falso negativo que só apareceu porque o grupo testou as duas formas.
- O Checkov não classifica severidade na versão gratuita, o que inviabiliza um gate por criticidade com essa ferramenta.
- O código de saída dos scripts empacotados do ZAP não reflete a severidade dos alertas, o que exigiu migrar para o Automation Framework.

- Alertas passivos do ZAP não são determinísticos entre execuções, o que torna instável qualquer gate construído sobre eles.

A recomendação de toolchain que decorre dessas medições é colocar Semgrep e Checkov no estágio Code, em todo pull request, pelo custo baixo e retorno imediato; o Dependency-Check no Build, com a base de vulnerabilidades em cache; e o ZAP no Test, com gate por severidade declarado no plano de automação. O gate que quebra o build por criticidade deve residir nas ferramentas que atribuem severidade real, o que, nesta toolchain, exclui o Checkov na versão gratuita.

9. Referências

Fontes primárias

1. CHECKOV. *What is Checkov*. Disponível em: <https://www.checkov.io/1.Welcome/What%20is%20Checkov.html>
2. CHECKOV. *CLI Command Reference*. Disponível em: <https://www.checkov.io/2.Basics/CLI%20Command%20Reference.html>
3. CHECKOV. *Suppressing and Skipping Policies*. Disponível em: <https://www.checkov.io/2.Basics/Suppressing%20and%20Skipping%20Policies.html>
4. CHECKOV. *Custom Policies Overview*. Disponível em: <https://www.checkov.io/3.Custom%20Policies/Custom%20Policies%20Overview.html>
5. ZAP. *ZAP Baseline Scan*. Disponível em: <https://www.zaproxy.org/docs/docker/baseline-scan/>
6. ZAP. *ZAP Full Scan*. Disponível em: <https://www.zaproxy.org/docs/docker/full-scan/>
7. ZAP. *Automation Framework: exitStatus Job*. Disponível em: <https://www.zaproxy.org/docs/desktop/addons/automation-framework/job-exitstatus/>
8. SEMGREP. *Semgrep Documentation*. Disponível em: <https://semgrep.dev/docs/>
9. SEMGREP. *Writing Rules: Overview*. Disponível em: <https://semgrep.dev/docs/writing-rules/overview>
10. OWASP. *Dependency-Check*. Disponível em: <https://owasp.org/projects/dependency-check>
11. DEPENDENCY-CHECK. *Documentation*. Disponível em: <https://jeremylong.github.io/DependencyCheck/>

12. DEPENDENCY-CHECK. *Analyzers*. Disponível em: <https://jeremylong.github.io/DependencyCheck/analyzers/index.html>

Fontes secundárias

1. OWASP. *OWASP Top Ten*. Disponível em: <https://owasp.org/www-project-top-ten/>
2. OWASP. *Source Code Analysis Tools*. Disponível em: https://owasp.org/www-community/Source_Code_Analysis_Tools
3. MITRE. *CWE-79: Improper Neutralization of Input During Web Page Generation*. Disponível em: <https://cwe.mitre.org/data/definitions/79.html>
4. MITRE. *CWE-284: Improper Access Control*. Disponível em: <https://cwe.mitre.org/data/definitions/284.html>
5. MITRE. *CWE-1021: Improper Restriction of Rendered UI Layers or Frames*. Disponível em: <https://cwe.mitre.org/data/definitions/1021.html>
6. NIST. *National Vulnerability Database*. Disponível em: <https://nvd.nist.gov/vuln>
7. CISA. *Software Bill of Materials (SBOM)*. Disponível em: <https://www.cisa.gov/sbom>
8. CYCLONEDX. *Specification Overview*. Disponível em: <https://cyclonedx.org/specification/overview/>
9. GITHUB. *SARIF support for code scanning*. Disponível em: <https://docs.github.com/en/code-security/code-scanning/integrating-with-code-scanning/sarif-support-for-code-scanning>
10. CROWDSTRIKE. *Static Application Security Testing (SAST)*. Disponível em: <https://www.crowdstrike.com/en-us/cybersecurity-101/cloud-security/static-application-security-testing-sast/>
11. CROWDSTRIKE. *Software Composition Analysis (SCA)*. Disponível em: <https://www.crowdstrike.com/en-us/cybersecurity-101/cloud-security/software-composition-analysis/>
12. FORTINET. *Dynamic Application Security Testing (DAST)*. Disponível em: <https://www.fortinet.com/br/resources/cyberglossary/dynamic-application-security-testing>
13. ZAP. *Automate Security Testing with ZAP and GitHub Actions*. Disponível em: <https://www.zaproxy.org/blog/2020-04-09-automate-security-testing-with-zap-and-github-actions/>
14. GITHUB. *bridgecrewio/checkov*. Disponível em: <https://github.com/bridgecrewio/checkov>